Case Study 9: Student Marks Matrix Analysis

1. Problem Statement

Store marks for multiple students and subjects in a nested list. Calculate each student's total and average, each subject's average, and identify the top performer.

2. Objectives

Represent data using nested lists.

Calculate row-wise and column-wise statistics.

Use loops to identify the top-performing student.

3. Assumptions

There are three students and three subjects.

Marks are between 0 and 100.

The student with the highest total is the top performer.

4. Algorithm

1. Start.

2. Input three student names.

3. Input marks for three subjects for each student.

4. Calculate each student's total and average.

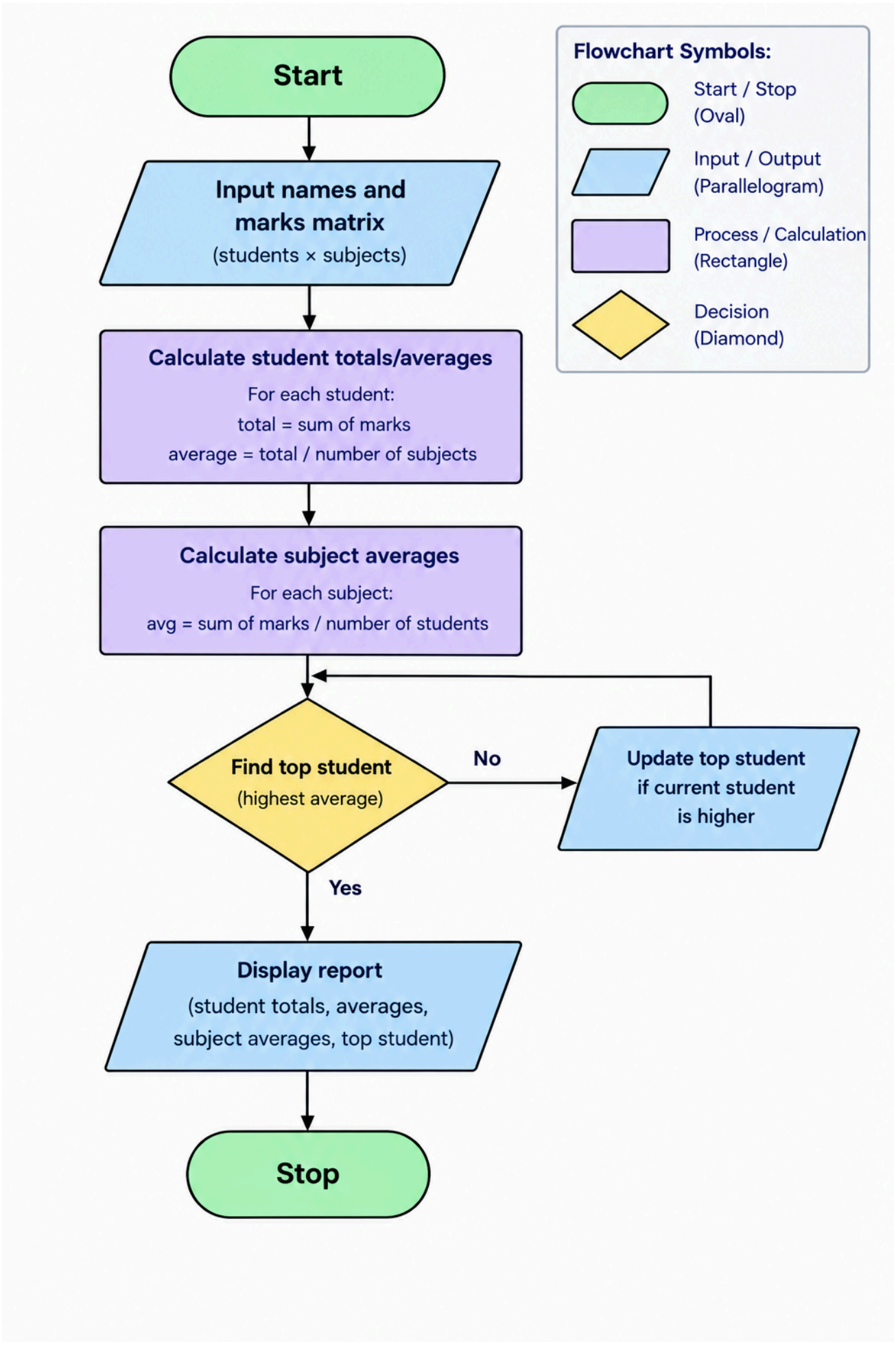
5. Calculate each subject's average.

6. Find the student with the highest total.

7. Display the analysis.

8. Stop.

5. Flowchart (text form)


Start
Input names and marks matrix
(students × subjects)
Calculate student totals/averages
For each student:
total = sum of marks
average = total / number of subjects
Calculate subject averages
For each subject:
avg = sum of marks / number of students
Find top student
(highest average)
No
Update top student if current student is higher
Yes
Display report
(student totals, averages, subject averages, top student)
Stop
Flowchart Symbols:
Start / Stop (Oval)
Input / Output (Parallelogram)
Process / Calculation (Rectangle)
Decision (Diamond)

6. Pseudocode

```
START
INPUT 3 student names
INPUT a 3 × 3 marks matrix
FOR each student row: calculate total and average
FOR each subject column: calculate average
Find index of maximum student total
DISPLAY student and subject statistics
STOP
```

7. Complete Python Source Code

```
students = []
marks_matrix = []

for i in range(3):
 name = input(f"Enter student {i + 1} name: ")
```



```
row = []
  for j in range(3):
mark = float(input(f"Enter marks for subject {j + 1} (0-100): "))
  while mark < 0 or mark > 100:
   print("Marks must be between 0 and 100.")
mark = float(input(f"Re-enter marks for subject {j + 1}: "))
row.append(mark)
students.append(name)
marks_matrix.append(row)

totals = []
  print("
  ----- STUDENT RESULTS -----")
  for i in range(3):
total = sum(marks_matrix[i])
  average = total / 3
totals.append(total)
  print(f"{students[i]}: total = {total:.1f}, average = {average:.2f}")
  print("
  ----- SUBJECT AVERAGES -----")
  for j in range(3):
subject_total = 0
  for i in range(3):
subject_total += marks_matrix[i][j]
  print(f"Subject {j + 1} average: {subject_total / 3:.2f}")

top_index = totals.index(max(totals))
  print("
  Top performer:", students[top_index])
  print("Highest total:", totals[top_index])
```

8. Test Case and Expected Output

Students: Asha [80, 70, 90]; Bala [75, 85, 80]; Charan [60, 65, 70].

Asha: total = 240.0, average = 80.00 Bala: total = 240.0, average = 80.00 Charan: total = 195.0, average = 65.00 Subject 1 average: 71.67 Subject 2 average: 73.33 Subject 3 average: 80.00 Top performer: Asha Highest total: 240.0

9. Conclusion

The program uses a nested list to analyse marks by student and subject, and reports the top performer. In a tie, the first student with the highest total is displayed.